

MANUAL TÉCNICO

GymPOS — Sistema de Punto de Venta para Gimnasio

Índice

1. Introducción
2. Objetivos del Sistema
3. Alcance del Proyecto
4. Tecnologías Utilizadas
5. Arquitectura del Sistema
6. Patrón MVC Implementado
7. Estructura de Packages
8. Descripción de Módulos
9. Diseño de Clases
10. Persistencia de Datos
11. Manejo de Excepciones
12. Multithreading
13. Interfaz Gráfica
14. Flujo General del Sistema
15. Diagramas UML
16. Instalación y Configuración
17. Pruebas del Sistema
18. Conclusiones
19. Bibliografía

1. Introducción

GymPOS es una aplicación de escritorio desarrollada en Java utilizando JavaFX, cuyo propósito es administrar de manera eficiente un gimnasio mediante un sistema de punto de venta y control administrativo.

El sistema integra múltiples funcionalidades como gestión de clientes, administración de membresías, control de acceso, generación de reportes, control de inventario y calendario de clases grupales.

Este proyecto fue desarrollado aplicando los conceptos vistos durante el semestre, incluyendo Programación Orientada a Objetos, arquitectura MVC, serialización de datos, manejo de excepciones, concurrencia e interfaces gráficas.

El objetivo principal del sistema es automatizar procesos administrativos de un gimnasio, reduciendo errores manuales y mejorando la organización de la información.

2. Objetivos del Sistema

Objetivo General

Desarrollar un sistema de escritorio que permita administrar de manera eficiente las operaciones principales de un gimnasio.

Objetivos Específicos

- Implementar un sistema CRUD para clientes.
- Gestionar membresías y planes de suscripción.
- Simular procesamiento de pagos.
- Llevar control de acceso al gimnasio.
- Administrar inventario de equipos.
- Mostrar calendario de clases grupales.
- Generar reportes automáticos.

- Aplicar arquitectura MVC.
- Implementar persistencia mediante serialización.
- Utilizar multithreading para tareas secundarias.

3. Alcance del Proyecto

El sistema GymPOS permite cubrir las necesidades básicas de administración de un gimnasio.

Entre las funcionalidades implementadas se encuentran:

- Registro de clientes.
- Edición y eliminación de clientes.
- Asignación de membresías.
- Validación de membresías activas.
- Sistema de puntos para clientes.
- Registro de accesos.
- Inventario de equipos.
- Calendario visual de clases.
- Generación de reportes.
- Persistencia de información mediante archivos .dat.

El sistema está diseñado para funcionar de manera local como aplicación de escritorio.

4. Tecnologías Utilizadas

Tecnología	Descripción
Java	Lenguaje principal del sistema
JavaFX	Interfaz gráfica
FXML	Diseño visual de vistas
CSS	Estilos visuales
VS Code	Entorno de desarrollo
Scene Builder	Diseño visual de interfaces
Serialización	Persistencia de datos
GitHub	Control de versiones

Java

Java fue utilizado como lenguaje principal debido a su robustez, orientación a objetos y compatibilidad multiplataforma.

JavaFX

JavaFX fue utilizado para la construcción de la interfaz gráfica, permitiendo crear interfaces modernas e interactivas.

FXML

FXML permitió separar el diseño visual de la lógica del sistema.

5. Arquitectura del Sistema

El sistema fue desarrollado utilizando arquitectura MVC (Model View Controller).

Modelo (Model)

Contiene las clases que representan los datos y entidades del sistema.

Ejemplos:

- Cliente
- Membresia
- Plan
- Equipo
- Acceso

Vista (View)

Contiene las interfaces gráficas desarrolladas con FXML.

Ejemplos:

- main.fxml
- membresias.fxml
- inventario.fxml
- calendario.fxml

Controlador (Controller)

Contiene la lógica de interacción entre el usuario y el sistema.

Ejemplos:

- MainController
- MembresiaController
- InventarioController
- DashboardController

6. Patrón MVC Implementado

El patrón MVC fue utilizado para mantener una estructura organizada y modular.

Ventajas del MVC

- Separación de responsabilidades.
- Mayor mantenibilidad.
- Código más organizado.
- Facilidad para agregar nuevas funcionalidades.
- Reutilización de componentes.

Funcionamiento

El usuario interactúa con la Vista.

La Vista envía eventos al Controller.

El Controller procesa la lógica utilizando Services y Models.

Finalmente, los datos son actualizados nuevamente en la Vista.

7. Estructura de Packages

El proyecto se encuentra organizado mediante packages.

Package app

Contiene la clase principal App.java encargada de iniciar el sistema.

Package controller

Contiene todos los controladores de las vistas.

Package model

Contiene las entidades del sistema.

Package service

Contiene la lógica de negocio.

Package util

Contiene clases auxiliares como el serializador.

Package view

Contiene los archivos FXML de las interfaces.

8. Descripción de Módulos

Gestión de Clientes

Permite:

- Registrar clientes.
- Actualizar información.
- Eliminar clientes.
- Mostrar clientes en tabla.
- Persistir información.

Sistema de Membresías

Permite:

- Asignar membresías.
- Validar membresías activas.
- Gestionar planes.
- Controlar vencimientos.

Procesador de Pagos

Simula pagos de membresías.

También asigna puntos de recompensa a los clientes.

Generador de Reportes

Genera reportes automáticos con información de clientes.

Control de Acceso

Permite verificar si un cliente puede ingresar al gimnasio.

Inventario de Equipos

Permite administrar:

- Equipos disponibles.
- Equipos en mantenimiento.
- Equipos fuera de servicio.

Calendario de Clases

Muestra clases grupales mediante un calendario visual.

9. Diseño de Clases

Clase Cliente

Representa a los clientes registrados.

Atributos

- id
- nombre
- email
- puntos

Métodos

- getters y setters
- constructor

Clase Membresía

Representa una membresía asignada.

Atributos

- cliente
- plan
- fechaInicio
- fechaFin

Clase Plan

Representa los diferentes planes del gimnasio.

Ejemplos

- Mensual
- Trimestral
- Anual

Clase Equipo

Representa equipos del gimnasio.

Atributos

- nombre
- cantidad
- estado

10. Persistencia de Datos

La persistencia del sistema fue implementada utilizando serialización.

Serialización

Permite guardar objetos Java directamente en archivos.

Los datos son almacenados en archivos con extensión .dat.

Ejemplos:

- clientes.dat
- membresias.dat
- inventario.dat

Ventajas

- Fácil implementación.
- No requiere base de datos.
- Persistencia rápida.
- Adecuado para proyectos académicos.

Clase Serializador

La clase Serializador contiene métodos para:

- guardar objetos.
- cargar objetos.

11. Manejo de Excepciones

El sistema implementa manejo de excepciones para evitar errores durante la ejecución.

Validaciones implementadas

- Campos vacíos.
- Datos inválidos.
- Cantidades incorrectas.
- Selecciones nulas.
- Membresías duplicadas.

Objetivo

Garantizar estabilidad y evitar fallos del sistema.

12. Multithreading

El sistema implementa multithreading para tareas secundarias.

Ejemplo implementado

Verificación automática de vencimiento de membresías.

Funcionamiento

Se ejecuta un hilo independiente que revisa continuamente las membresías registradas.

Cuando una membresía vence, el sistema muestra una notificación automática.

Ventajas

- Mejor rendimiento.
- Procesos en segundo plano.
- Interfaz más fluida.

13. Interfaz Gráfica

La interfaz fue desarrollada utilizando JavaFX y FXML.

Características

- Diseño modular.
- Navegación mediante menú lateral.
- Tablas dinámicas.
- Formularios interactivos.

- Calendario visual.
- Diseño responsivo.

Vistas principales

- Dashboard
- Gestión de Clientes
- Membresías
- Inventario
- Acceso
- Calendario

14. Flujo General del Sistema

Inicio del sistema

La clase App.java inicia la aplicación.

Navegación

El usuario accede a diferentes módulos mediante el Dashboard.

Gestión de información

Los datos son procesados por Services.

Persistencia

La información se guarda automáticamente en archivos .dat.

15. Diagramas UML

Diagrama de Casos de Uso

Actores principales:

- Administrador
- Cliente

Casos de uso:

- Registrar cliente.
- Asignar membresía.
- Registrar acceso.
- Generar reporte.
- Administrar inventario.

Diagrama de Clases

Clases principales:

- Cliente
- Membresia
- Plan
- Equipo
- Acceso

Relaciones:

- Cliente posee Membresia.
- Membresia utiliza Plan.
- Controllers utilizan Services.

16. Instalación y Configuración

Requisitos

- JDK 17 o superior.
- JavaFX SDK.
- Visual Studio Code.
- Extension Pack for Java.

Configuración JavaFX

Se debe configurar el module-path de JavaFX.

Ejemplo:

```
--module-path ruta/javafx/lib --add-modules javafx.controls,javafx.fxml
```

Ejecución

1. Abrir el proyecto en VS Code.
2. Ejecutar App.java.
3. Iniciar el sistema.

17. Pruebas del Sistema

Se realizaron pruebas funcionales sobre:

Clientes

- Registro.
- Actualización.
- Eliminación.

Membresías

- Asignación.
- Validación.
- Vencimientos.

Inventario

- Registro de equipos.
- Cambios de estado.
- Persistencia.

Acceso

- Validación de membresía activa.

Reportes

- Generación correcta de archivos.

18. Conclusiones

El desarrollo de GymPOS permitió aplicar múltiples conceptos fundamentales de programación orientada a objetos y desarrollo de aplicaciones de escritorio.

Durante el proyecto se implementaron estructuras modulares, persistencia de datos, interfaces gráficas, multithreading y arquitectura MVC.

El sistema cumple con los requerimientos principales planteados inicialmente y representa una solución funcional para la administración básica de un gimnasio.

Además, el proyecto permitió fortalecer habilidades en Java, JavaFX y diseño de software.

19. Bibliografía

Oracle. (2024). Java Documentation. Recuperado de:

<https://docs.oracle.com>

OpenJFX. (2024). JavaFX Documentation. Recuperado de:

<https://openjfx.io>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994).

Design Patterns: Elements of Reusable Object-Oriented Software.

Addison-Wesley.

Wikipedia. (2024). Modelo Vista Controlador.

Recuperado de:

<https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador>

Deitel, P., & Deitel, H. (2017).

Java: Cómo Programar.

Pearson.